Provider and License Management

Executive Summary · Senior Full-Stack .NET Modernization Developer take-home assignment

Dionicio Gonzalez · October 10, 2026 · Branch `feature/provider-license-solution`

The solution is a working web application for state staff to manage child care providers and their licenses. A React user interface calls an ASP.NET Core Web API, which stores data in SQLite through Entity Framework Core. It meets every required item in the assignment, includes the optional dashboard, and enforces the organization's central rule, **no record is ever physically deleted**, in the database itself as well as in the application.

What Was Delivered

- **Provider management:** list with search, filters and paging; create; edit; delete with a confirmation step.
- **Licenses:** each provider's licenses, showing the stored status beside the validity calculated from the expiration date.
- **Issue License:** a new license, with a generated number, for a provider that has no valid one.
- **Required scenarios:** active providers with their valid licenses; active providers with expired licenses; deleted providers excluded everywhere.
- **Dashboard:** summary cards, bar and pie charts, and expiring licenses. Every number opens the records behind it.
- **Audit view:** soft-deleted providers, with who deleted them and when.
- **Database deliverables:** the SQLite file with sample data, and SQL scripts for the schema, the scenarios, and the sample data.
- **Automated tests:** 174 for the API and database, 130 for the user interface.

Architecture



Each layer has one job. The interface never decides whether a license is valid or which records a report contains; it shows what the API returns. The API never trusts the interface; it validates every request again. The database never trusts the API; it rejects invalid rows on its own.

Technology Stack

LAYER	TECHNOLOGY	WHY
User interface	React 19, JavaScript, Vite 8	Required by the assignment; kept as supplied by the starter. No UI, routing or charting library was added.
API	ASP.NET Core Web API, C#, .NET 8	Required. .NET 8 matches the starter, so reviewers need nothing extra installed.
Data access	Entity Framework Core 8, SQLite provider	Required. Migrations build the whole database, including triggers and views.
Database	SQLite	Required. A single file, so the solution runs as cloned with no server.
Tests	xUnit; Vitest and Testing Library	Standard, widely known tools for each side of the stack.

Technical Approach and Reasoning

Soft delete is enforced in layers, not by convention. A delete sets a flag, a timestamp and the user; the row stays. Four independent safeguards back this up: a query filter hides deleted rows from every application query automatically; a save interceptor converts any delete issued in code into a soft delete; database triggers reject a physical `DELETE` even from a SQL tool; and a check constraint keeps the flag and timestamp consistent. *Reasoning:* a rule that depends on every developer remembering a `WHERE` clause will eventually be broken. Here a single mistake is caught by another layer.

Business rules live in the API, and the data rules live in the database. Name length, allowed statuses, required relationships and uniqueness are database constraints. Duplicate checks, license eligibility and date rules are in API services. *Reasoning:* the organization maintains real systems where scripts, reports and other applications touch the same data. Rules kept only in the user interface, or only in application code, do not protect against those.

License validity is calculated from the expiration date, in one place. A license is expired once its date has passed, whatever its stored status says, and a provider's own status is never used to judge its licenses. "Today" is the US Eastern date, supplied by a replaceable clock. *Reasoning:* stored statuses fall behind the calendar; the sample data includes a license still marked Active after it expired. A single definition, used by the lists, reports and dashboard alike, means they cannot contradict each other.

Concurrent edits are detected, not silently overwritten. Each record carries a stamp that changes on every save; an update must present the stamp it loaded. *Reasoning:* with several staff editing the same providers, the second save should be told, not lose the first.

Uniqueness works with soft delete. A provider name is unique within a county among live records only, so a deleted provider does not block entering a replacement. *Reasoning:* a plain unique index would make deleted rows permanently reserve their names.

Errors are structured and safe to show. The API returns standard problem responses: field-level messages for validation, and clear messages for not-found and conflict. The interface shows them beside the relevant field and handles loading, empty and failure states on every screen. *Reasoning:* support staff and users need to know what to correct, and internal details should never reach the browser.

Tests run against a real SQLite database. Backend tests build an in-memory SQLite database with the same migrations as the application, so constraints, triggers and views are exercised, including attempts to bypass the rules with raw SQL. *Reasoning:* the common shortcut, an in-memory substitute, enforces none of those and would have tested the least important parts.

Few dependencies, and a design that can grow. The charts are plain CSS and SVG, and navigation is component state. Authentication is absent, as in the assignment, but audit columns are filled through a single replaceable component. *Reasoning:* every added package is something to patch and upgrade; every seam left in place makes the next requirement cheaper.

Trade-offs and Next Steps

- **No authentication.** Audit columns record a placeholder user, and the audit view is open.
- **SQLite allows one writer at a time.** A multi-user system would move to SQL Server or Azure SQL; the design carries over.
- **License workflow is limited to issuing.** Edit, suspend and renew are not built.
- **Next:** authentication and role-based access, a restore operation, and a full change-history table.
- **Next:** upgrade to .NET 10 before .NET 8 support ends in November 2026.
- **Next:** a build pipeline running both test suites, and TypeScript or generated API types.

Full setup instructions, the API reference and database design are in `README.md`. The alternatives considered for each decision are in `docs/DECISIONS.md`.